Singly Linked List – Delete At A Position –Space Complexity and Algorithm

Program:

```
void deleteAtPosition(Node **head, int position)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        Node *previous = nullptr;
        int i = 1;
        while (i < position && current != nullptr)
        {
            previous = current;
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            if (previous == nullptr)
            {
                *head = current->next;
            }
            else
            {
                previous->next = current->next;
            }
          cout << "Deleted node: " << current->data << endl;
          current->next = nullptr;
          free(current);
         current = nullptr;
          previous = nullptr;
          cout << "Node is deleted successfully." << endl;
        }
    }
}

…. .

   case 8:
        {
            cout << "Delete at a position,indexing node from 1 to N:" << endl;
            int position;
            cout << "Enter the node position to delete: ";
            cin >> position;
            deleteAtPosition(&head, position);
            break;
        }
```

Space Complexity

Input Space:

A. Parameter – Only Definition

The input space consists of the parameters passed to the function: Node ** head and int position.

Both of these require constant space, i.e., O(1), because they do not depend on the size of the linked list.

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int position` (an integer), both of which have a fixed, constant size. Therefore, the input space complexity is **O(1)**.

- ***Node **head (one machine word)***
  - ***int position (one machine word)***

  ⇒ Θ(1) = O(1)

B. Full – Data (inclusive) Definition

This more common definition includes the entire data structure. If the linked list has **n nodes** before the deletion, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

- ***Same two machine words as above (Θ(1)), plus***
  - ***Every node that can be reached from *head when the function is called.***

    If the list contains n nodes, they occupy Θ(n) bytes.

  ⇒ Θ(n) = O(n)

Auxiliary Space:

The auxiliary space is the additional space used by the function during its execution.

In this function, the auxiliary space is primarily used for temporary variables:

- ***Node * current: A pointer used to traverse the linked list.***
- ***Node * previous: A pointer used to keep track of the previous node during traversal.***
- ***int i: An integer used as a loop counter.***

Each of these variables requires constant space, i.e., O(1), because they do not depend on the size of the linked list.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`Node *current`**, **`Node *previous`**, and **`int i`**: The function declares two pointers and one integer to traverse the list and keep track of the position. Each of these variables occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses these three variables to find the correct node. No new memory is allocated during the traversal. This affects the function's **time complexity** (making it O(k), where k is the position), but not its space complexity.
3. **`free(current)`**: This function *releases* memory back to the system; it does not consume extra working space for the algorithm.

Since the function only needs a fixed number of extra variables (two pointers and an integer) to do its job, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

• Local stack variables:

– Node *current

– Node *previous

– int i

– a few compiler-generated temporaries

All of these together are constant-size.

• No additional dynamic allocations (the only heap operation is free).

• No recursion.

⇒ Θ(1) = O(1)

Algorithm

$DELATPOS(START, POS)$

1. $[LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ and\ EXIT.$

$[End\ Of\ If\ Structure]$

2. $[POS < 1\ ?\ i.e.\ INDEX\ POSITION\ STARTS\ FROM\ 1]$

$if\ POS < 1\ , then:$

$Write:$ `` $Error: Invalid\ Position.$ ``

$Return\ and\ exit.$

$[Delete\ Node\ , When\ POS \geq 1\ ]$

$ELSE$

$SET\ CURRPTR := START\ \begin{bmatrix} CURRPTR\ is\ a\ pointer\ set\ to\ START, and\ will\ store \\ address\ of\ Deleted\ Node. \end{bmatrix}$

$SET\ PREVPTR := NULL\ \begin{bmatrix} PREVPTR\ is\ a\ pointer\ set\ to\ NULL\ and\ will\ store \\ address\ of\ PREVIOUS\ NODE\ of\ DELETED\ NODE. \end{bmatrix}$

$SET\ COUNT := 1[Why\ `1`\ as\ indexing\ starts\ from\ `1`\ here.]$

$\begin{bmatrix} THIS\ THE\ LOOP\ , When\ POSITION > 1\ , SET\ PREVPTR\ to\ PREVIOUS\ address\ of \\ DELETED\ NODE\ and\ CURRPTR\ to\ address\ of\ DELETED\ NODE. \end{bmatrix}$

$Repeat\ while\ COUNT < POS\ AND\ CURRPTR \neq NULL\ then:$

$SET\ PREVPTR := CURRPTR\ \begin{bmatrix} SET\ PREVPTR\ to\ the\ address\ , that\ CURRPTR \\ points\ to. \end{bmatrix}$

$SET\ CURRPTR := LINK[CURRPTR] \begin{bmatrix} SET\ CURRPTR\ to\ the\ address\ that \\ LINK\ of\ the\ NODE\ holds\ , and\ this \\ NODE\ to\ which\ CURRPTR\ is\ pointing. \end{bmatrix}$

$SET\ COUNT := COUNT + 1\ [INCREMENT\ COUNT]$

$[END\ OF\ LOOP]$

$\begin{bmatrix} When\ POSITION\ is\ input\ by\ USER\ which\ is\ ``out\ of\ bounds.``\ i.e.\ exceed\ to\ no.\ of\ nodes \\ existed. \end{bmatrix}$

$if\ CURRPTR = NULL\ , then:$

$Write : ``\ Error: INVALID\ POSITION``$

$Return\ and\ exit$

$[When\ POSITION\ input\ by\ USER\ which\ is\ not\ ``out\ of\ bounds.``]$

$ELSE$

$[When\ POSITION\ is\ 1\ and\ PREVPTR\ , then\ will\ remain\ at\ NULL]$

$if\ PREVPTR = NULL\ , then:$

$SET\ START := LINK[CURRPTR]$ $\begin{bmatrix} SET\ START\ or\ HEAD\ to\ LINK\ of\ NODE\ that \\ CURRPTR\ points\ to\ i.e.\ NULL, which \\ delinks\ HEAD\ from\ that\ single\ NODE. \end{bmatrix}$

$[When\ POSITION > 1\ and\ PREVPTR\ \neq\ NULL]$

$ELSE$

$SET\ LINK[PREVPTR] := LINK[CURRPTR]$ $\begin{bmatrix} Sets\ LINK\ of\ PREVIOUS\ NODE\ to \\ address\ hold\ by\ LINK\ of\ NODE \\ pointed\ by\ CURRPTR, which \\ makes\ the\ PREVIOUS\ NODE \\ points\ to\ that\ address. \end{bmatrix}$

$[END\ of\ If\ Structure]$

$Write: "Deleted\ node: "\ followed\ by\ INFO[CURRPTR]$ $\begin{bmatrix} Print\ the\ data\ of\ the\ node \\ being\ deleted. \end{bmatrix}$

$SET\ LINK[CURRPTR] := NULL\ [SET\ LINK\ of\ NODE\ pointed\ by\ CURRPTR\ to\ NULL]$

$FREE(CURRPTR)[Deallocate\ the\ memory\ used\ by\ the\ node.]$

$SET\ CURRPTR := NULL\ [Sets\ Dangling\ pointer\ CURRPTR\ to\ NULL]$

$SET\ PREVPTR := NULL\ [Sets\ Dangling\ pointer\ PREVPTR\ to\ NULL]$

$Write: "Node\ is\ deleted\ successfully."\ [Write\ Statement\ on\ Confirmation\ of\ Deletion]$

$[END\ OF\ IF\ STRUCTURE]$